Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-380: $4^{η}$ Σειρά Ασκήσεων

• Άσκηση $1^{η}$:

α) Κύριες διαφορές μεταξύ BFS (Breadth-first search) και DFS (Depth-first search):

1. Διάσχιση: ο αλγόριθμος BFS εξερευνά τον γράφο επίπεδο προς επίπεδο, ελέγχοντας όλους τους γείτονες ενός κόμβου πριν προχωρήσει πιο βαθιά - ενώ ο αλγόριθμος DFS εξερευνά όσο το δυνατόν πιο βαθιά σε ένα μονοπάτι μέχρι να βρει αδιέξοδο, και μετά κάνει οπισθοδρόμηση.
2. Δομή Δεδομένων: ο αλγόριθμος BFS χρησιμοποιεί ουρά για να κρατάει τη σειρά των κόμβων προς επίσκεψη - ενώ ο αλγόριθμος DFS χρησιμοποιεί (αναδρομική) στοίβα.
3. Αποτέλεσμα: ο αλγόριθμος BFS εγγυάται την εύρεση του συντομότερου μονοπατιού σε γράφους χωρίς βάρη - ενώ ο αλγόριθμος DFS δεν εγγυάται την εύρεση του συντομότερου μονοπατιού, απλώς βρίσκει ένα οποιοδήποτε μονοπάτι.

β) Σε ποιές περιπτώσεις χρησιμοποιούμε τον έναν και σε ποιες τον άλλον αλγόριθμο:

1. Ο αλγόριθμος BFS είναι ιδανικός όταν ξέρουμε ότι η λύση μας βρίσκεται σχετικά κοντά στην ρίζα ή όταν θέλουμε να βρούμε το συντομότερο μονοπάτι. Για παράδειγμα, εύρεση συντομότερου μονοπατιού ενός λαβύρινθου ή αναζήτηση στα κοινωνικά δίκτυα.
2. Ο αλγόριθμος DFS είναι ιδανικός όταν θέλουμε να εξερευνήσουμε όλες τις πιθανές λύσεις σε ένα πρόβλημα, όταν θέλουμε να επισκεφτούμε όλους τους κόμβους γρήγορα, ή όταν το πρόβλημα απαιτεί οπισθοδρόμηση. Για παράδειγμα, λύση ενος λαβύρινθου ή εύρεση κύκλων σε γράφο ή εύρεση τοπολογικής διάταξης/ταξινόμησης.

γ) Το ίδιο για κατευθυνόμενους γράφους:

0. Η λογική των αλγορίθμων (και άρα η απάντηση) παραμένει ακριβώς η ίδια, με τη μόνη διαφορά ότι η διάσχιση επιτρέπεται αυστηρά μόνο προς την κατεύθυνση των ακμών ...

• Άσκηση $2^{η}$:

⋆ Αλγόριθμος που να βρίσκει τη διάμετρο (μεγαλύτερο από ελάχιστα μονοπάτια) του γράφου:
⋆ Σημείωση: υποθέτουμε πως ο γράφος είναι μη σταθμισμένος και συνεκτικός ...

0. Αλγόριθμος Εύρεσης Διαμέτρου Γράφου:
1. Αρχικοποιούμε μια μεταβλητή για την τιμή της διαμετρου με μηδέν.
2. Για κάθε κόμβο στο σύνολο κορυφών:
3. Εκτελούμε τον αλγόριθμο BFS με αρχή τον κόμβο αυτό.
4. Αποθηκεύουμε τις συντομότερες αποστάσεις (σε πλήθος ακμών) από τον κομβο προς όλους τους άλλους.
5. Βρίσκουμε και κρατάμε τη μέγιστη απόσταση που ανακαλύφθηκε σε αυτή την αναζήτηση.
6. Αν η μέγιστη απόσταση είναι μεγαλητερη από την τρέχουσα διάμετρο:
7. Τότε ενημερώνουμε τη μεταβλητή της διαμέτρου με την μέγιστη απόσταση.
8. Μετά την ολοκλήρωση του βρόχου, επιστρέφουμε την τιμή της μεταβλητής διαμέτρου.
9. Τέλος.

⋆ Χρονική Πολυπλοκότητα: εκτέλεση BFS για κάθε κόμβο, άρα $\mathcal{O}(V(V+E))$ ...

• **Άσκηση 3η:**

⋆ Αλγοριθμος ΠιοΑξιόπιστοΜονοπάτι (G, r, S, D):
// G: ο κατευθυνόμενος γράφος (V, E).
// r: η συνάρτηση βάρους/αξιοπιστίας όπου 0 <= r(u,v) <= 1.
// S: κόμβος εκκίνησης & D: κόμβος προορισμού.
Για κάθε κόμβο v στο σύνολο V:
Prob[v] = 0.0 // Αρχικά, η πιθανότητα να φτάσουμε είναι 0 (πίνακας για τη μέγιστη πιθανότητα).
Prev[v] = null // Δεν υπάρχει προηγούμενος κόμβος ακόμα (πινακας για ανακατασκευή του μονοπατιού).
Prob[S] = 1.0 // Η πιθανότητα να βρισκόμαστε στην αφετηρία είναι 100%.
Q = new(empty max priority queue) // Αρχικοποίση μιας άδειας ουράς Q που είναι Empty Max-Priority Queue.
Q.insert(S, Prob[S]) // Εισαγωγή του κόμβου εκκίνησης στην ουρά.
Όσο η ουρά Q δεν είναι άδεια:
u = Q.extractMax() // Εξαγωγή του κόμβου u με τη μεγαλύτερη τιμή Prob[u].
Αν u == D: Break // Αν φτάσαμε στον προορισμό, μπορούμε να τερματίσουμε νωρίτερα.
Για κάθε γειτονικο κόμβο v του u:
new_prob = Prob[u] ∗ r(u, v) // Υπολογίζουμε τη νέα πιθανότητα αν πάμε μέσω του u.
Αν new_prob > Prob[v]:
Prob[v] = new_prob // Ενημέρωση με την καλύτερη (μεγαλύτερη) πιθανότητα.
Prev[v] = u // Καταγραφή του u ως προηγούμενου κόμβου του v.
Αν v υπάρχει στην Q: Q.increaseKey(v, Prob[v]) // Ενημέρωση της ουράς προτεραιότητας.
Αλλιώς: Q.insert(v, Prob[v]) // −||−
Path = new(empty list) // Λίστα για να δημιουργήσουμε το μονοπατι (εάν υπάρχει).
current = D
Αν Prob[D] == 0.0 και S ! = D: // Αν το Prob[D] είναι 0, σημαίνει ότι ο D δεν είναι προσβάσιμος από τον S.
Return 0.0, «Δεν βρέθηκε μονοπάτι»
Όσο current δεν είναι null:
Path.prepend(current) // Προσθήκη στην αρχή της λίστας.
current = Prev[current] // Ενημέρωση του δείκτη.
Return (Prob[D], Path) // Επιστρέφουμε το ποιο αξιόπιστο μονοπάτι...
⋆ Τέλος.

• **Άσκηση 4η:**

⋆ Αλγόριθμος Μεταβατική Κλειστότητα Κατευθυνόμενο Γράφου (G):
// Έστω n το πλήθος των κορυφών (|V|).
T[n][n] = False // Δημιούργησε έναν δισδιάστατο πίνακα και αρχικοποιουμε όλα τα στοιχεία του.
Για κάθε κορυφή u στο G.V:
T[u][u] = True // Κάθε κορυφή μπορεί να φτάσει στον εαυτό της.
Q = new(empty queue
Q.enqueue(u)
Όσο η ουρά Q δεν είναι κενή:
v = Q.dequeue()
Για κάθε γείτονα w της κορυφής v:
Αν T[u][w] == False: // Αν δεν έχουμε ήδη βρει μονοπάτι από την αρχική κορυφή u προς την w.
T[u][w] = True // Σημειώνουμε ότι η w είναι προσβάσιμη από την u.
Q.enqueue(w) // Προσθέτουμε την w στην ουρά για να εξερευνήσουμε τους γείτονές της.
Return (T)
ΤΕΛΟΣ

⋆ Πολυπλοκότητα: ο συγκεκριμένος αλγόριθμος υπολογίχει την μεταβατική κλειστότητα ενος κατευθυνόμενου γράφου. Αρχικά, η αρχικοποίηση του δισδυάστατου πίνακα T απαιτεί χρόνο O(|V| ∗ |V|)=O(|V^2|), αφού ελέγχει όλους τους κόμβους. Έπειτα, σε κάθε επανάληψη του εξωτερικού βρόχου, εκτελουμε αναζήτηση κατά πλάτος, όπου κάθε κόμβος προστίθεται στην ουρά το πολύ μία φορά, και άρα οι ακμές που εξετάζονται θα είναι το πολύ |E|. Η αναζήτηση αυτή θα πραγματοποιηθεί για κάθε κόμβο, έτσι χρειάζεται χρόνος |V|∗O(|V| + |E|) = O(|V|^2 + |V| ∗ |E|). Τελικά: $O(|V|^2) + O(|V|^2 + |V| \cdot |E|) = O(|V|^2 + |V| \cdot |E|)$. Συμπέρασμα: άν ο γράφος δεν αποτελείται μόνο από απομονωμένες κορυφές (δηλαδή αν $|E| \geq |V|$), ο όρος $|V| \cdot |E|$ κυριαρχεί του $|V|^2$, και επομένως, η συνολική χρονική πολυπλοκότητα είναι $O(|V| \cdot |E|)$, δηλαδή ακριβώς η ίδια με το ζητουμενο ...

• **Άσκηση 5η:**

⋆ Ο κλασικός αλγόριθμος Kruskal έχει χρονική πολυπλοκότητα $O(|E| \log |V|)$ και σε αυτό ευθύνεται ο αλγόριθμος ταξινομησης που θα χρησιμοποιηθεί. Εάν υποθέσουμε ότι όλα τα βάρη των ακμών του γράφου είναι ακέραιοι αριθμοί και παίρνουν τιμές απο 1 μέχρι |V| ή από 1 μέχρι τον σταθερο θετικό ακέραιο W, θα πρέπει να επιλέξουμε τον γρηγορότερο αλγοριθμό ταξηνόμησης, έτσι ωσστε η χρόνική πολυπλοκότητα να γίνει βέλτιστη. Δεν γνωρίζω ποιοι είναι αυτοί η αλγόριθμοι...

• **Άσκηση 6η:**

α) Αλγόριθμος για εύρεση του maximum spanning tree - MST σε έναν γράφο $G = (V, E)$ με συνάρτηση βάρους w:

1. Τροποποίηση βαρών: δημιουργούμε έναν νέο γράφο $G' = (V, E)$, ο οποίος είναι ολόιδιος με τον G, αλλά η μόνη διαφορά είναι ότι η νέα συνάρτηση βάρους w' προκύπτει πολλαπλασιάζοντας τα αρχικά βάρη με το -1.
2. Εκτέλεση κλασικού αλγορίθμου: εκτελούμε έναν γνωστό και αποδεδειγμενο αλγόριθμο εύρεσης ελάχιστου επικαλύπτοντος δέντρου (όπως τον αλγόριθμο του Kruskal ή Prim) πάνω στον νέο γράφο G'.
3. Επιστροφή αποτελέσματος: το δέντρο T που θα επιστρέψει ο αλγόριθμος, αποτελεί το μέγιστο επικαλύπτον δέντρο του αρχικού γράφου G. Για το τελικό αποτέλεσμα, απλώς επαναφέρουμε τα αρχικά θετικά βάρη $w(e)$ στις ακμές του δέντρου.

β) Απόδειξη ορθότητας αλγορίθμου:

1. Θεωρούμε δεδομένο ότι ο κλασικός αλγόριθμος που χρεισιμοποιηθηκε λειτουργεί ορθά. Συνεπώς, θα βρει με βεβαιότητα το δέντρο T που ελαχιστοποιεί το συνολικό άθροισμα των τροποποιημένων βαρών w' στον γράφο G': $W'(T) = \min \sum_{e \in T} w'(e)$.
2. Όμως, εφόσον έχουμε αντιστρέψει όλα τα βάρη, αντικαθιστουμε το $w'(e)$ με $(-w(e))$, και έτσι, η παραπάνω εξίσωση γίνεται: $W'(T) = \min \sum_{e \in T} -w(e)$.
3. Το δέντρο T που ελαχιστοποιεί το βάρος στον G', είναι ακριβώς το ίδιο δέντρο που μεγιστοποιεί το συνολικό άθροισμα $\sum_{e \in T} w(e)$ των αρχικών βαρών στον γράφο G, αφού ισχύει: $W'(T) = \min \left(- \sum_{e \in T} w(e) \right) \equiv - \max \sum_{e \in T} w(e)$.

∗) Τελικά, ο αλγόριθμός μας βρίσκει όντως το μέγιστο επικαλύπτον δέντρο του γράφου − και σε χρόνο $O(E \log V)$ ...

• **Άσκηση 7η:**

∗ Αλγόριθμος ο οποίος βρίσκει μια βέλτιστη κάλυψη n κόμβων (vertex cover) ενός δέντρου ...

1. Διαλέγουμε τυχαία έναν κόμβο του δέντρου για να είναι η ρίζα.
2. Δημιουργούμε ένα κενό σύνολο, το οποίο στο τέλος θα περιέχει τους κόμβους της βέλτιστης κάλυψης.
3. Εκτελούμε μια μεταδιατεταγμένη διάσχιση, έτσι ώστε να επεξεργαζόμαστε τον κάθε κόμβο αφού πρώτα έχουμε ελέξει όλα τα παιδιά του (απο κάτω προς τα πάνω).
4. Για κάθε κόμβο του δέντρου που εξετάζουμε ελέγχουμε:
5. Αν έχει τουλάχιστον ένα παιδί το οποίο δεν ανήκει στο σύνολο, τότε πρέπει υποχρεωτικά να τον προσθέσουμε (ώστε να καλυφθεί η ακμή μεταξύ τους).
6. Αλλιώς αν όλα τα παιδιά του ανήκουν ήδη στο σύνολο (ή αν δεν έχει κανενα παιδί), τότε δεν τον προσθέτουμε.
7. Τελειώνοντας, επιστρέφουμε το σύνολο που περιέχει πλέον μια βέλτιστη κάλυψη του δέντρου ...

∗ Πολυπλοκότητα ...

1. Χρονική: ο αλγόριθμος βασίζεται σε μία απλή διάσχιση κατά πλατος. Κάθε κόμβος επισκέπτεται μία φορά και κάθε ακμή, από τις ακριβώς $n - 1$ του δέντρου, ελέγχεται σταθερό αριθμό φορών. Έτσι, ο συνολικός χρόνος εκτέλεσης είναι $O(n)$.
2. Χωρική: απαιτείται μνήμη για την αποθήκευση του σύνολου (στη χειρότερη περίπτωση περιέχει $n - 1$ κόμβους), καθώς και για τη στοίβα της αναδρομής (χειρότερη περιπτωση φτάνει σε βάθος n. Άρα η μνήμη θα είναι $O(n)$.

• **Άσκηση 8η:**

α. Απόδειξη Ύπαρξης Euler Tour:
(⇒) Αν ο G έχει Euler tour, τότε in-degree(v) = out-degree(v)): Ενα Euler tour είναι πρακτικά ένας κύκλος που περνάει από κάθε ακμή του γράφου ακριβώς μία φορά. Για κάθε κόμβο $v \in V$, κάθε φορά που το τουρ εισέρχεται στον v μέσω μιας ακμής, πρέπει αναγκαστικά να εξέλθει από τον v μέσω μιας άλλης, αχρησιμοποίητης ακμής. Αφού το tour ξεκινάει και καταλήγει στον ίδιο κόμβο, και συμπεριλαμβάνει όλες τις ακμές του G, κάθε εισερχόμενη ακμή σε έναν κόμβο αντιστοιχίζεται μοναδικά με μία εξερχομενη ακμή. Επομένως, για κάθε κόμβο v, ο αριθμός των εισερχόμενων ακμών είναι ίσος με τον αριθμό των εξερχόμενων, δηλαδή in-degree(v) = out-degree(v).
(⇐) Αν in-degree(v) = out-degree(v) για κάθε v, τότε ο G έχει Euler tour: Ξεκιναμε από έναν τυχαίο κόμβο και ακολουθούμε αχρησιμοποίητες ακμές. Επειδή κάθε κόμβος έχει ίσες εισερχόμενες και εξερχόμενες ακμές, όποτε μπαίνουμε σε έναν κόμβο υπάρχει πάντα διαθέσιμη ακμή για να βγούμε, οπότε δεν μπορούμε να εγκλωβιστούμε σε κανέναν κόμβο εκτός από τον αρχικό. Αυτή η διαδρομή αναγκαστικά θα κλείσει σχηματίζοντας έναν κύκλο. Αν ο κύκλος αυτός δεν περιέχει όλες τις ακμές του γράφου, τότε θα υπάρχει τουλάχιστον ένας κόμβος σε αυτόν τον κύκλο που έχει αχρησιμοποίητες ακμές. Ξεκινώντας από αυτόν τον κόμβο, επαναλαμβάνουμε τη διαδικασία για να βρούμε έναν νέο κυκλο και τον συγχωνεύουμε με τον αρχικό, αντικαθιστώντας τον κόμβο με τον νέο κύκλο. Επαναλαμβάνοντας αυτή τη διαδικασία συγχώνευσης, θα συμπεριλάβουμε τελικά όλες τις ακμές του γράφου, σχηματίζοντας το ζητούμενο Euler tour.

• **Άσκηση 9η:**

α. Αποδείξη ότι ρίζα γραφου είναι ένα σημείο άρθρωσης αν και μόνο αν έχει τουλάχιστον δύο παιδιά:
∗ Αν η ρίζα έχει μόνο ένα παιδί, τότε το γράφημα μπορεί να εξερευνηθεί πλήρως από αυτό το μοναδικό παιδί, αλλά αν αφαιρεθεί η ρίζα, το υπόλοιπο δέντρο παραμένει συνδεδεμένο, οπότε η ρίζα δεν είναι σημείο άρθρωσης.
∗ Αν η ρίζα έχει δύο ή περισσότερα παιδιά, τότε επειδη δεν υπάρχουν ακμές διασταύρωσης, δεν υπάρχει κανένα μονοπάτι που να συνδέει το υποδέντρο του ενός παιδιού με το υποδέντρο του άλλου χωρίς να περάσει από τη ρίζα, και έτσι η αφαίρεση της ρίζας αποσυνδέει αυτά τα υποδέντρα μεταξύ τους - άρα είναι σημείο άρθρωσης.

β. Αποδειξη ότι εσωτερικός κόμβος που δεν ειναι ρίζα του γράφου είναι ένα σημείο άρθρωσης αν και μόνο αν έχει ένα παιδί τέτοιο ώστε δεν υπάρχει back edge από αυτο ή οποιοδήποτε απόγονό του σε έναν πρόγονό του:
∗ Αν δεν υπάρχει back edge, τότε είναι σημείο άρθρωσης, αφού εστω ότι ο v έχει ένα παιδί s τέτοιο ώστε ούτε το s ούτε κανένας απόγονος του s έχει back edge προς κάποιον πρόγονο του v. Επειδή, όπως είπαμε, δεν υπάρχουν cross edges, οποιοδήποτε μονοπάτι από το υποδέντρο του s προς το υπόλοιπο γράφημα πρέπει να περάσει υποχρεωτικά από τον v, άρα η αφαίρεσή του θα απομονώσει εντελώς το υποδέντρο του s, κάνοντας τον v σημείο άρθρωσης.
∗ Αν είναι σημείο άρθρωσης, τότε δεν υπάρχει τέτοια back edge, αφου αν για κάθε παιδί του v υπάρχει τουλάχιστον μία back edge από το υποδέντρο του παιδιού προς έναν πρόγονο του v, τότε η αφαίρεση του v δεν αποσυνδέει τα παιδιά του από το υπόλοιπο γράφημα, αφού διατηρείται μια εναλλακτική διαδρομή προς τα πάνω, και έτσι, σε αυτή την περίπτωση, ο v δεν θα ήταν σημείο άρθρωσης.

ε. Απόδειξη ότι μια ακμή ενος γράφου είναι μια γέφυρα, αν και μόνο αν δεν βρίσκεται σε οποιοδήποτε απλό κύκλο του γράφου:
∗ Αν η ακμή δεν ανήκει σε κύκλο, είναι γέφυρα, αφού εστω μια ακμή (u, v) η οποία δεν ανήκει σε κανέναν απλό κύκλο. Αυτό σημαίνει ότι αποτελεί το μοναδικό μονοπάτι μεταξύ των κόμβων u και v, άρα αν την αφαιρέσουμε, δεν υπάρχει άλλη διαδρομή για να φτάσουμε από τον u στον v, οπότε το γράφημα αποσυνδέεται, και η ακμή είναι όντως γέφυρα.
∗ Αν η ακμή ανήκει σε κύκλο, δεν είναι γέφυρα, αφού έστω ότι η ακμή (u, v) ανήκει σε έναν απλό κύκλο. Γνωρίζουμε πως ενας κύκλος παρέχει δύο διαφορετικά μονοπάτια μεταξύ οποιωνδήποτε δύο κόμβων του, άρα αν αφαιρεθεί, χρησιμοποιώντας τον υπόλοιπο κύκλο, μπορούμε να μεταβούμε απο τον u στον v, και έτσι το γράφημα δεν αποσυνδέεται από την αφαίρεσή της, οπότε δεν είναι γέφυρα.

LaTeX